Segunda auditoría de software y production readiness

Repositorio AELB · 19 de agosto de 2026 · Re-verificación de la auditoría del 18 de agosto de 2026

> 🔴 **NO LISTO PARA PRODUCCIÓN**

PRODUCTION READINESS: 52/100 (auditoría anterior: 42/100)

1. Conclusión ejecutiva

Los tres bloqueantes CRITICAL de la auditoría anterior han tenido trabajo real: **C-01 y C-02 están corregidos y confirmados con tests dedicados** (`SecurityConfigAuthorizationTest`). C-03 sigue abierto. Pero esta segunda auditoría ha encontrado algo más grave que cualquier hallazgo de la lista anterior: **el backend no compila sus tests, y como consecuencia la imagen Docker del backend no se puede construir en absoluto ahora mismo**. Se ha comprobado ejecutando `docker build` real contra el `Dockerfile` del repositorio — falla. No es una sospecha: es un hecho reproducible, documentado en el §19.

La causa es una regresión: un trabajo en curso de paginación SQL real (`UserRepository.findPage(...)`) cambió una interfaz de dominio, y un stub de test (`EmailServiceAdapterTest.java`) quedó desincronizado. Es un fix de una línea, pero hoy, en este commit, no existe forma de producir un artefacto backend desplegable ni de ejecutar ninguno de los ~35 tests del proyecto — incluidos los que demuestran que C-01/C-02 están arreglados.

2. Cambios desde la auditoría anterior

- Matcher específico y anterior en orden para `/api/championships/*/registrations/pdf` (C-01).
- `PUT /api/users/*/admin` restringido a `hasRole("ADMIN")`; GESTOR ya no puede tocarlo (C-02).
- Reserva de inscripción **antes** de ir a Stripe, no después (H-02).
- Session binding, locks pesimistas, validación de importe/usuario/año e idempotencia en confirmación de cuota (H-03).
- Logging con `correlationId` correlacionable en el webhook de Stripe (H-06, mejora parcial).
- `FileUploadValidator` con magic bytes reales para documentos/resultados/PDF (H-04, parcial).
- Sistema completo de recuperación de contraseña/verificación de email: outbox transaccional, worker con retry+backoff+rate limit, SES en prod / Mailpit en local con validación fail-fast (H-05, prácticamente nuevo).
- `application-prod.properties` + `EmailConfigurationValidator` que impide arrancar en `prod` con secretos de email faltantes o con clave de cifrado de ejemplo.
- Actuator con liveness/readiness separados correctamente; graceful shutdown configurado.
- Suite de tests backend ampliada de "muy limitada" a ~18 clases de test.
- Framework de tests frontend (Vitest + Testing Library, 19 tests) añadido — no existía antes.
- **Regresión nueva**: el cambio de paginación rompió la compilación de tests y, con ello, el build Docker del backend.

3. Verificación C-01 → C-03

✅ C-01 — PDF de inscripciones público — SOLUCIONADO, CONFIRMADO

`SecurityConfig.java:99-105`: matcher específico `GET /api/championships/*/registrations/pdf → hasAnyRole("GESTOR","ADMIN")` declarado **antes** del `permitAll()` genérico (línea 105). Doble protección: `RegistrationController.java:206-209` repite el chequeo de rol manualmente. Test dedicado: `SecurityConfigAuthorizationTest.registrationPdfRequiresAuthentication` (anónimo → 401). Hueco de cobertura: no hay test que compruebe 403 para rol USER autenticado. No bloquea.

✅ C-02 — GESTOR podía promover a ADMIN — SOLUCIONADO, CONFIRMADO

`SecurityConfig.java:157`: `PUT /api/users/*/admin → hasRole("ADMIN")` (ya no `hasAnyRole`). El único DTO con campo `role` (`AdminUpdateUserRequest`) solo se usa en esa ruta; `ManagementUpdateUserRequest` (la que GESTOR sí puede usar) no tiene campo `role` ni `blocked`. Test dedicado: `gestorCannotChangeRoleOrBlockedStatus` → 403. No bloquea.

🔴 C-03 — Docker Compose con configuración local — SIGUE PRESENTE

Un único `docker-compose.yml`, sin separación dev/prod: `SPRING_PROFILES_ACTIVE: local` hardcodeado (línea 122), `AUTH_COOKIE_SECURE:-false` (línea 125), MySQL publicado `3307:3306`, MinIO publicado `9000/9001` con imágenes `latest` y credenciales `minioadmin` por defecto. El *código* de la app (perfil `prod`, `S3Config` con IAM Role, `EmailConfigurationValidator`) ya está bien preparado — el problema es que nada activa automáticamente esa configuración; sigue dependiendo al 100% de que un humano lo haga bien a mano. Bloquea.

4. Verificación H-01 → H-08

ID	Estado	Evidencia clave
H-01 Migraciones manuales	🔴 **SIGUE (mitigado)**	Sin Flyway/Liquibase. `schema.sql` verificado coherente con las 13 migraciones. Ninguna es destructiva; la de `birth_date NOT NULL` falla intencionadamente y de forma segura. No bloquea BD nueva; 100% manual en BD existente.
H-02 Pago sin inscripción	✅ **SOLUCIONADO**	`CreateStripeChampionshipBasketCheckoutUseCase` reserva la inscripción PENDING antes de crear la sesión Stripe; el webhook solo transiciona PENDING→PAID. Ya no existe el escenario "cobrado sin inscribir".
H-03 Webhook cuota sin session binding	✅ **SOLUCIONADO (gap menor)**	Lock pesimista + `@Transactional` + búsqueda por `stripe_session_id` UNIQUE + validación usuario/año/importe + idempotencia vía `isPaid()`. Gap: basket/registro no revalidan importe/divisa del webhook (mitigado, el total lo fija el propio backend de forma inmutable).
H-04 Uploads sin límites	🟡 **PARCIAL**	Documentos/resultados: magic bytes + límite real ✅. Noticias/campeonatos: S3 con whitelist ✅. Nuevo hallazgo: logo de equipos sin ningún límite ni validación (ver N-03).
H-05 Password reset / email	✅ **SOLUCIONADO**	Outbox transaccional + worker con retry exponencial+jitter, tokens hasheados+expiración+un solo uso, no-enumeración con trabajo dummy, cooldown y máximo/hora server-side, SES/Mailpit por perfil con validación fail-fast.
H-06 Stripe errores silenciosos	🟡 **MEJORADO**	`correlationId` en cada línea de log; evento/sessionId logueados en la recepción. Ya no es caja negra.
H-07 Sin CI/CD	🔴 **SIGUE**	Sin `.github/workflows` ni equivalente, nunca ha existido en el historial de git.

H-08 Frontend lint falla	🔴 **SIGUE, SIN CAMBIOS**	**16 errores, 22 warnings — número idéntico a la auditoría anterior.** `next build` sí compila.

5. Verificación M-01 → M-08

ID	Estado	Evidencia
M-01 Aforo	🔴 **Confirmado ausente (no es bug)**	Sin `capacity` / `maxParticipants` en dominio ni esquema.
M-02 Concurrencia inscripciones	🟡 **Mitigado por BD**	Read-check-write sin lock explícito, protegido por UNIQUE de BD.
M-03 Security headers	🔴 **SIGUE**	Sin CSP ni Referrer-Policy explícitos; solo defaults de Spring Security.
M-04 Rate limiting en memoria	ℹ️ **No bloqueante**	Una sola instancia de backend — aceptable a este volumen.
M-05 Paginación	🟡 **PARCIAL**	Listado normal con SQL real; filtro por cuota sigue en memoria.
M-06 Límites Base64	🔴 **SIGUE, inconsistente**	Documentos/resultados con límite; `Team.logoBase64` sin ninguno.
M-07 Borrado de usuarios	🔴 **Confirmado — 500 genérico**	FK RESTRICT sin `@ExceptionHandler` dedicado; sin pérdida de datos, sí UX rota.
M-08 Vulnerabilidades npm	✅ **SOLUCIONADO**	`npm audit --omit=dev` → 0 vulnerabilidades.

6. Nuevos hallazgos desde la auditoría anterior

N-01 — `@PreAuthorize` sin efecto por falta de `@EnableMethodSecurity` · HIGH (latente) · CONFIRMADO

`PricingController` / `ChampionshipPricingController` usan `@PreAuthorize`, pero no existe `@EnableMethodSecurity` en el proyecto — son código muerto. No explotable hoy porque `SecurityConfig` respalda esas rutas con matchers equivalentes, pero es el mismo patrón que causó C-01.

N-02 — Backend no compila sus tests / imagen Docker no se puede construir · CRITICAL · CONFIRMADO (ejecutado)

`EmailServiceAdapterTest.java:24` — el stub `UserRepositoryAdapterStub` no implementa los nuevos `findPage(int,int)` / `findPage(int,int,String,String)` de `UserRepository.java:25-26`. Confirmado con `mvn clean verify` real y con `docker build` real — ambos fallan, incluso con `-DskipTests`. Ver §19.

N-03 — Logo de equipos: sin límite/validación + XSS almacenado vía Content-Type · MEDIUM-HIGH · CONFIRMADO

`CreateTeamRequest` / `UpdateTeamRequest`: `logoBase64` / `logoType` sin ninguna anotación de validación. `TeamController.getLogo` (endpoint público) hace `MediaType.parseMediaType(team.getLogoType())` con el valor declarado por el cliente. Un GESTOR podría subir `logoType=text/html` con payload malicioso; cualquier visitante que abra `/api/teams/{id}/logo` lo recibiría como HTML ejecutable.

N-04 — Falta `.dockerignore` en ambos módulos · MEDIUM

El frontend hace `COPY . .` sin filtro; riesgo de incluir `.env` / `.git` en el contexto de build.

N-05 — Sin `HEALTHCHECK` en Dockerfiles/compose · LOW-MEDIUM

Pese a que Actuator ya expone liveness/readiness correctamente configurados.

N-06 — `AUTH_COOKIE_SECURE` con default `false` incluso en el perfil base · LOW-MEDIUM

`application.properties:31`. Refuerza por qué C-03 sigue siendo real: la seguridad es opt-in, no por defecto.

N-07 — `CancelRegistrationUseCase` rechaza toda cancelación incondicionalmente · INFO, verificar intención

`CancelRegistrationUseCase.java:19-21` lanza excepción sin mirar quién llama ni el estado. Puede ser una política de negocio deliberada, pero hoy es indistinguible de un bug.

7. Seguridad

Autenticación: JWT en cookie HttpOnly, `Secure` configurable (N-06), `SameSite=Strict` por defecto, refresh token rotatorio hasheado, `tokenVersion` que revoca sesiones. CSRF: patrón doble-cookie coherente con el uso del frontend. Autorización: auditoría IDOR/BOLA completa sobre los 15 controladores — sin hallazgos explotables más allá de C-01/C-02 (corregidos) y N-01 (latente). Contenido no publicado correctamente filtrado. CORS con lista blanca, sin wildcard. Frontend: sin `dangerouslySetInnerHTML`, único `NEXT_PUBLIC_*` es la URL de API, sin tokens en `localStorage`.

8. Pagos Stripe

Firma HMAC-SHA256 en tiempo constante + tolerancia de replay 300s. Idempotency-Key determinista al crear sesiones. Los tres flujos de confirmación son `@Transactional` con lock pesimista e idempotencia de negocio vía `isPaid()`. Session binding por ID único en BD. Importe/usuario/año validados en cuota; basket/registro no revalidan importe del webhook (mitigado). Sin aforo modelado (confirmado, no bug si no se requiere). Cobertura de test (leída, no ejecutada): cubre pago válido, metadata de otro usuario, importe manipulado. No cubre: firma inválida, sesión desconocida, session mismatch, webhook retrasado, idempotencia explícita, fallo de materialización.

9. Base de datos y migraciones

Esquema con FKs, UNIQUE y NOT NULL correctos. Sin Flyway/Liquibase; `schema.sql` verificado coherente con las 13 migraciones (comprobación estática contra entidades JPA). Ninguna migración destructiva. Índice recomendable ausente: `quotas(year, payment_status)`.

10. Concurrencia

Doble inscripción y cuota duplicada protegidas por UNIQUE de BD, no solo por check en Java. `RegisterForChampionshipUseCase` es read-check-write sin lock explícito pero con la constraint como red de seguridad final (no verificado si la excepción de integridad resultante se traduce a un mensaje de dominio legible).

11. Email

Outbox transaccional con recuperación de jobs abandonados. Worker con rate limiter no bloqueante, retry con backoff exponencial+jitter, excepciones permanentes vs temporales. Simulando 10.000 emails pendientes: se procesan en lotes respetando `maxPerSecond`, no se envían simultáneamente. SES solo activable en perfil `prod` (validación fail-fast); Mailpit en local/dev. Ni el token de reset ni el de verificación aparecen en ningún log.

12. Observabilidad

`CorrelationIdFilter` añade `X-Correlation-ID` a toda respuesta y al MDC; patrón de log lo incluye en cada línea. Liveness/readiness correctamente separados (liveness no depende de BD/proveedores externos). Graceful shutdown configurado. No verificado en runtime (la app no se ha llegado a arrancar en esta auditoría de solo lectura) — correcto sobre el papel, confirmado por código.

13. Frontend

Sin XSS vía `dangerouslySetInnerHTML`. CSRF integrado en todas las llamadas mutantes. Manejo de errores centralizado sin exponer stack traces. Único env var expuesto no sensible. Nada de credenciales en `localStorage`.

14. Backend

Arquitectura hexagonal consistente. Nota de mantenibilidad: parte de la infraestructura más nueva (email, Stripe webhook) está escrita en estilo muy condensado (clases en una línea), contrastando con el resto del código bien formateado — dificulta la revisión sin ser un problema funcional.

15. Tests

Unit/dominio: sí. Seguridad/API (MockMvc): sí. Stripe: sí, parcial. Uploads: sí. Email: sí. Concurrencia real, migraciones, E2E: no existen. **Ninguno se ha podido ejecutar en esta auditoría** — el módulo no compila. Bug crítico que podría entrar ahora sin que ningún test lo detecte: cualquier regresión en inscripción/Stripe/SecurityConfig, porque el pipeline que lo pillaría no corre — ni local de forma fiable, ni en CI (inexistente).

16. CI/CD

No existe. Confirmado por búsqueda en el árbol actual y en todo el historial de git — nunca ha existido.

17. Docker / AWS

Backend Dockerfile: multi-stage, no-root, imagen fijada por versión. **Build real ejecutado: FALLA** (mismo error de N-02). Frontend Dockerfile: mismo patrón correcto. **Build real ejecutado: ÉXITO**, no-root confirmado. Ninguno tiene `HEALTHCHECK` ni `.dockerignore`. AWS: `S3Config` soporta IAM Role condicionado a `STORAGE_USE_IAM=true`, sin credenciales hardcodeadas. `.env` real (no trackeado, en `.gitignore`, nunca commiteado) contiene clave Stripe de **test**, no de producción.

18. Backups / Rollback

Sin estrategia de backup recurrente documentada. Sin restauración probada nunca. Sin procedimiento de rollback más allá de git.

19. Checks ejecutados

Check	Resultado
Maven build (compile, JDK 21 real)	✅ **PASS — 323 ficheros, sin errores**
Maven test-compile / `mvn clean verify`	🔴 **FAIL — stub no implementa `findPage`**
Backend tests (surefire)	⛔ **NO EJECUTADOS — bloqueados**
Docker build backend	🔴 **FAIL — mismo error, incluso con `-DskipTests`**
Docker build frontend	✅ **PASS — no-root confirmado**
Frontend `npm ci`	✅ **PASS**
Frontend `npm run lint`	🔴 **FAIL — 16 errores, 22 warnings (idéntico a antes)**
Frontend `npm run build`	✅ **PASS — 27 rutas**
Frontend tests (`vitest run`)	✅ **PASS — 19/19**
`npm audit --omit=dev`	✅ **PASS — 0 vulnerabilidades**
DB migrations	✅ **PASS (razonamiento estático, sin MySQL real)**
Security tests (leídos)	⛔ **NO EJECUTADOS — correctos por lectura estática**
Stripe tests (leídos)	⛔ **NO EJECUTADOS — cobertura parcial**

20. Tabla completa de hallazgos

ID	Estado	Evidencia	Comentario
C-01	✅	SecurityConfig.java:99-105 + test	Solucionado y probado
C-02	✅	SecurityConfig.java:157 + test	Solucionado y probado
C-03	🔴	docker-compose.yml:40,64,122,125	Sin cambios
H-01	🔴 **(mitigado)**	Sin Flyway; schema.sql coherente	No bloquea BD nueva
H-02	✅	Reserva antes de pagar	—
H-03	✅ **(gap menor)**	Lock+transacción+session binding	Falta validar importe en basket/registro
H-04	🟡	FileUploadValidator	Nuevo hueco: logo de equipos (N-03)
H-05	✅	Outbox+worker+SES/Mailpit	Prácticamente nuevo
H-06	🟡	correlationId en logs	Ya no es caja negra
H-07	🔴	Sin .github/workflows	Sin cambios
H-08	🔴	16 errores lint	Sin cambios, mismo número
M-01	🔴 **(no es bug)**	Sin capacity en dominio	—
M-02	🟡	Protegido por UNIQUE	Sin lock explícito
M-03	🔴	Sin CSP/Referrer-Policy	—
M-04	ℹ️	Una instancia	No bloquea a este volumen
M-05	🟡	findPage parcial	Filtro por cuota en memoria

M-06	🔴	Límites inconsistentes	Team.logoBase64 sin límite
M-07	🔴	FK RESTRICT + 500 genérico	Sin pérdida de datos, sí UX rota
M-08	✅	npm audit limpio	—
N-01	⚠️ **nuevo**	@PreAuthorize inerte	No explotable hoy
N-02	⚠️ **nuevo CRITICAL**	Build backend roto	Confirmado ejecutando
N-03	⚠️ **nuevo**	Logo equipo sin validar	XSS almacenado posible

21. Production readiness score

Área	Puntuación	Motivo
Seguridad	7/10	C-01/C-02 corregidos y probados; faltan headers y @EnableMethodSecurity; hueco en logo de equipos
Fiabilidad	6/10	Stripe genuinamente más fiable; sin red de tests activa hacia adelante
Base de datos	6/10	Buen esquema, coherente; sin herramienta de migración
Arquitectura	8/10	Hexagonal consistente, pagos/email bien diseñados
Calidad de código	3/10	Build roto; lint sin cambios; estilo inconsistente en módulos nuevos
Tests	3/10	Buena cobertura sobre el papel, cero ejecutable hoy
Observabilidad	7/10	correlationId, health separado, graceful shutdown — confirmado en código
Configuración	5/10	Perfil prod bien preparado; el compose que se usaría no lo activa
Deployment	2/10	Sin CI/CD y, ahora, sin poder construir la imagen backend
Mantenibilidad	5/10	Arquitectura clara; estilo condensado nuevo reduce legibilidad

PRODUCTION READINESS: 52/100

22. Bloqueantes actuales

BLOQUEANTES REALES PARA DESPLEGAR HOY:

1. El backend no compila sus tests y su imagen Docker no se puede construir (N-02) — arreglar el stub `UserRepositoryAdapterStub` en `EmailServiceAdapterTest.java` para implementar `findPage(int,int)` / `findPage(int,int,String,String)`.
2. `docker-compose.yml` sigue sin separación dev/prod (C-03) — perfil local forzado, cookies inseguras por defecto, puertos de MySQL/MinIO publicados, credenciales por defecto.

Eso es todo lo que impide técnicamente el despliegue de este commit exacto. Todo lo demás es riesgo asumible o mejora, no bloqueo literal.

23. Riesgos aceptables v1

- **H-01** (sin Flyway) — asumible con disciplina manual documentada; revisar antes de crecer.

- **H-07** (sin CI/CD) — asumible para un lanzamiento manual cuidadoso una vez resuelto el bloqueante 1; no indefinidamente.
- **M-03** (sin CSP/HSTS explícitos) — asumible, barato de añadir en el reverse proxy al desplegar.
- **M-04** (rate limiter en memoria) — correcto para una sola instancia.
- **M-07** (borrado de usuario → 500) — molesto, no peligroso; la BD protege la integridad.
- **N-01** (@PreAuthorize inerte) — no explotable hoy; corregir antes de añadir endpoints que confíen en él.
- **N-03** (logo de equipo) — mitigado por ser rol de confianza; corregir antes de ampliar el número de gestores.

24. Top 3 incidentes probables

1. Desplegar con el compose actual sin overrides manuales completos. Probabilidad: media-alta. Impacto: alto (cookies no-Secure, posible exposición de MySQL/MinIO). Mitigación existente: ninguna automática. Acción pendiente: separar compose prod o forzar variables críticas.

2. Una cuenta GESTOR sube un logo de equipo malicioso. Probabilidad: baja. Impacto: medio-alto (XSS almacenado público). Mitigación existente: solo el rol de confianza. Acción pendiente: aplicar FileUploadValidator/whitelist a `Team.logoBase64`.

3. Una regresión futura en pago/inscripción pasa desapercibida por falta de tests ejecutándose. Probabilidad: media. Impacto: alto. Mitigación existente: los tests existen pero no corren. Acción pendiente: arreglar el bloqueante 1 y conectar CI mínimo con `mvn verify` en cada push.

25. Veredicto final

> ## 🔴 NO LISTO PARA PRODUCCIÓN

No por falta de progreso — hay progreso real y verificado en autorización, pagos y email — sino porque este commit exacto no produce un artefacto backend desplegable, y el compose que se usaría para desplegarlo sigue siendo inseguro por defecto. Ambos son arreglables en horas, no en semanas, pero hoy son bloqueantes reales, reproducidos por ejecución directa, no teóricos.

26. Deploy / no deploy

MI RECOMENDACIÓN: NO DEPLOY

Lista mínima exacta a resolver antes de desplegar:

1. Arreglar `EmailServiceAdapterTest.java` (implementar los métodos `findPage` que faltan en el stub) y confirmar `mvn clean verify` + `docker build -f backendweb/Dockerfile .` en verde.
2. Crear una configuración de despliegue real distinta del `docker-compose.yml` local: `SPRING_PROFILES_ACTIVE=prod`, `AUTH_COOKIE_SECURE=true` forzado, sin publicar puertos de MySQL/MinIO, sin credenciales por defecto.

Con solo esos dos puntos resueltos (ambos de alcance pequeño), y aceptando los riesgos del §23 como deuda técnica de v1, el veredicto pasaría razonablemente a 🟡 LISTO CON RIESGOS CONOCIDOS.